pfi

Python for Fixed Income

COURSE 1 · PYTHON FOUNDATIONS

pandas: Series and DataFrames, Part 1

WEEK 2, DAY 2, PART 1

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

By the end of this part you can

Build a Series and a DataFrame

A labelled column, and a table of them.

```
pd.Series(price_list, index=ticker_list)
```

Load a holdings file

Its size, its columns, their types, what is missing.

```
holdings = pd.read_csv(data_path)
```

Select columns

The two or three a question needs.

```
holdings[['ticker', 'coupon', 'price']]
```

Look up a bond

By position with iloc, by label with loc.

```
by_cusip.loc['99039NAC0', 'price']
```

From four bonds to two hundred

Week 1

4

bonds, 6 columns

→

Today

200

bonds, 19 columns

Each bond now comes with text (white) and numbers (dark)

CUSIP	Issuer name	Sector
Rating	Maturity date	Price
Spread	Duration	Position size

The tools so far, and the one for today

A list or a dictionary

Holds the data. Answers no question without a loop.

→

A NumPy array

Calculates on a whole column at once. Holds one type of value, and knows nothing about tickers or column names.

→

A pandas table

Text and numbers side by side. Every column with a name. Every row a bond.

Series: one labelled column. DataFrame: a table of them.

A Series is one labelled column

prices

index	values
QVNE	98.50
DNMR	103.25
PLWK	95.00
HLVF	101.00

dtype: float64

Built from two lists. Looked up by label, or by position.

```
prices = pd.Series(price_list, index=ticker_list)

print('PLWK:', prices['PLWK'])
PLWK: 95.0
print('First price:', prices.iloc[0])
First price: 98.5
```

Two arrays in different orders

Multiplied position by position. No error, and four wrong numbers.

Two Series in different orders

Lined up by ticker first, as a VLOOKUP would. A ticker missing on one side gives NaN, with no error.

A DataFrame is a table of Series

desk: four rows, six columns, shape (4, 6)

	Ticker	Coupon	Maturity	Price	Face	Rating
0	QVNE	5.250	2033	98.50	1500000	BBB
1	DNMR	6.125	2034	103.25	500000	BB+
2	PLWK	4.750	2031	95.00	750000	A-
3	HLVF	7.500	2032	101.00	250000	B+
dtype	str	float64	int64	float64	int64	str

Each column has one type, like an array. The table as a whole can mix them.

Every column is a Series

```
desk_prices = desk['Price']
print(type(desk_prices))
<class 'pandas.Series'>
```

In Excel

A worksheet with a header row.

The column names are the headers.

The index is the row numbers down the left edge.

The holdings file: 200 bonds, 19 columns

```
holdings = pd.read_csv(data_path)
```

One row per bond. The first line of the file becomes the column names. Eight of the nineteen columns are shown.

	cusip	ticker	sector	rating	coupon	price	oas	par_held
0	99080CAE8	BEZE	Financials	BBB-	7.000	106.727	147	5000000
1	99049XAE2	BEZH	Industrials	B	8.125	97.556	535	250000
2	99039NAC0	BEZL	Transportation	BB	5.125	98.730	153	1500000
units	text	text	text	text	percent of face	per 100 of face	basis points	dollars of face

`holdings.shape`

(200, 19): 200 bonds, 19 columns.

`holdings.info()`

200 non-null in every column. Nothing is missing.

```
holdings['oas']
# one name: a Series
holdings[['ticker', 'coupon', 'maturity', 'price']]
# a list of names: a smaller table
```

Rows: iloc by position, loc by label

```
by_cusip = holdings.set_index('cusip')
```

position	cusip (the index)	ticker	coupon	price
0	99080CAE8	BEZE	7.000	106.727
1	99049XAE2	BEZH	8.125	97.556
2	99039NAC0	BEZL	5.125	98.730

Sort the table or remove a bond, and position 0 is a different bond. A label stays with its row.

`iloc` **By position. A slice leaves out its end.**

`by_cusip.iloc[0:3]` positions 0, 1, 2

`loc` **By label. A slice includes its end.**

`by_cusip.loc['99080CAE8':'99039NAC0']` both ends

Question	pandas	Excel
Price of a CUSIP	`by_cusip.loc['99039NAC0', 'price']`	`=VLOOKUP("99039NAC0", B:I, 8, FALSE)`
A CUSIP that is not there	`KeyError`	`#N/A`

The AI check

Every notebook ends with code written in the style of an AI assistant. It has one bug. Find it.

1

Read

Read the code before you run it.

2

Predict

Decide what it should produce.

3

Run

Run it and look at the output.

4

Check

Compare with a number you worked out yourself.

Code that runs is not the same as code that is right.

What you can do now

- Build a Series and a DataFrame, and let the index line two columns up
- Load a holdings file and take a first look at its size, columns, and types
- Pull out the columns a question needs
- Look a bond up by its CUSIP with loc, and by its position with iloc

Next pandas: Series and DataFrames, Part 2: filter, rank, add to, and show the table